

$$\text{Sigmoid}(S) = \frac{1}{1 + e^{-x}}; \quad s' = s(1 - s)$$

$$\text{Sigmoid}(\sigma(x)) = \frac{1}{1 + e^{-x}}; \quad \sigma(x)' = \sigma(x)(1 - \sigma(x))$$

Chain rule:

$$u = 2x + 5$$

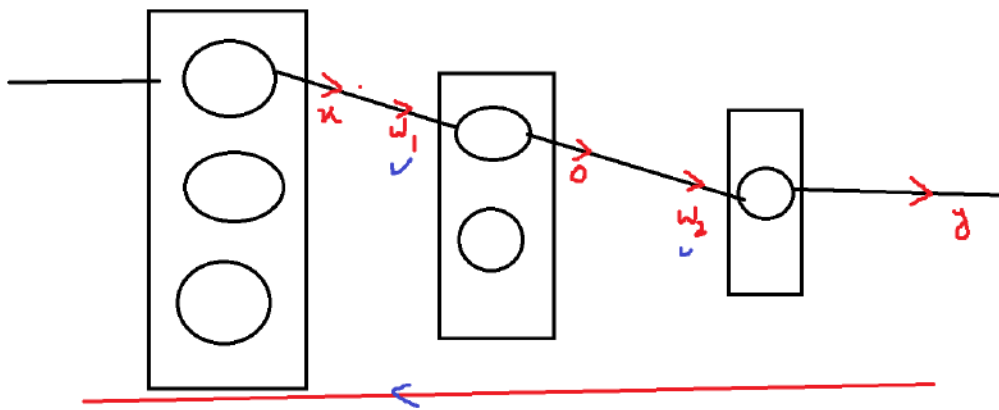
$$y = 3u + 4$$

$$\frac{dy}{dx} = \text{in } y \text{ equation } x \text{ available : No}$$

$$\frac{dy}{du} : \text{in } y \text{ equation } u \text{ available : yes}$$

$$\frac{du}{dx} : \text{in } u \text{ equation } x \text{ available : yes}$$

$$\frac{dy}{dx} = \frac{dy}{du} * \frac{du}{dx}$$



$$w_{1new} = w_{1old} - lr * \frac{dj}{dw_1}$$

$$w_{2new} = w_{2old} - lr * \frac{dj}{dw_2}$$

$$j = cost(y_{Ac} - y_p)$$

$$y_p = \sigma(w_2 * o + b_2)$$

$$j = \text{interms of } \sigma(w_2 * o + b_2)$$

$$\frac{dj}{dw_2} = \text{availabe}$$

$$\frac{dj}{dw_1} = \text{not availabe}$$

$$o = \sigma(w_1 * x + b_1)$$

$$j = cost(y_a - \sigma(w_2 * o + b_2))$$

$$\frac{dj}{dw_1} = \frac{dj}{d\sigma} * \frac{d\sigma}{do} * \frac{do}{dw_1}$$

$$\frac{d\sigma}{do} = \sigma * (1 - \sigma)$$

case - 1: when $\sigma = 0$ the diff value is $\sigma * (1 - \sigma) = 0$

case - 2: when $\sigma = 1$ the diff value is $\sigma * (1 - \sigma) = 0$

case - 2: when $\sigma = 0.5$ the diff value is $\sigma * (1 - \sigma) = 0.25$

$\sigma * (1 - \sigma)$ ranges from 0 to 0.25

assume that $\sigma * (1 - \sigma) = 0.25$

$$\frac{dj}{dw_1} = \frac{dj}{d\sigma} * \frac{d\sigma}{do} * \frac{do}{dw_1}$$

$$\frac{dj}{dw_1} = \frac{dj}{d\sigma} * 0.25 * \frac{do}{dw_1} \quad i - h - o$$

$$i - h - h - o : 0.25 * 0.25$$

$$i - h - h - h - o : 0.25 * 0.25 * 0.25 = 0.0001625$$

$$\frac{dj}{dw_1} = i - h - h - h - o \cong 0$$

- when Neural network hidden layers increase the differentiation of activation number increase
- every differentiation of sigmoid activation function has max value 0.25
- when we do more multiplies of 0.25 at one point of time the value will be approx 0
- assume 100 hidden layers : $(0.25)^{100} \cong 0$

$$w_{1new} = w_{1old} - lr * 0$$

$$w_{1new} = w_{1old}$$

- weights not updating
- no learning
- but still error is maximum
- this problem is called Vanish Gradients problem
- This happens because hidden layer neurons are using sigmoid and tanh activation function
- So Hidden layers will not use sigmoid tanh and softmax activation functions

- sigmoid , softmax and tanh all are facing Vanish gradients problem

- *sigmoid will use in output layer bi classification*
- *softmax will use in output layer multi classification*
- *tanh -1 to 1 if any use cases we need output -1 to 1 then go for tanh*
- *before ReLU introduces tanh used in hidden layers also*
- *Still LSTM RNN AutoEncoders are using tanh in hidden layers*
- *ReLU 0 to inf are used in hidden layers*
 - *ReLU assumes if inputs are negative it assumes as zero*
 - *What if all inputs are negative then ReLU converts all inputs into 0*
 - *in this situation we will use Leaky ReLU*

GPT models we use GELU in Hidden layers: Gaussian Error Linear unit

In output layer no activation, just use raw values that logits

$$\sigma(w_1 * x + b) = \text{activation on logits}$$

$$w_1 * x + b = \text{logit}$$

